

LinkFlow 项目架构与目录说明（标准化参考）

本文档面向首次接触本仓库的研发/测试/运维同学，用于快速理解 LinkFlow 的代码结构、运行入口、模块职责、依赖关系与后续维护的关键关注点。

1. 项目概览

LinkFlow 当前形态是一个“PC 端内核（Python）+ 前端控制台（静态 HTML/JS）”的本地局域网工具：

- **PC 内核（Python）**
 - 提供 **TCP Socket 服务**：面向手机端或其他客户端的自定义二进制协议通信。
 - 提供 **WebSocket 网桥**：面向 PC 浏览器页面的实时数据推送与指令回传。
 - 内置系统监控、剪贴板监听、文件目录列表等能力。
- **前端页面（web_pc / web_mobile）**
 - web_pc：通过 WebSocket 直连 PC 内核，用于仪表盘/文件浏览等。
 - web_mobile：当前为 UI/交互原型（mock 数据），未实际接入后端。

默认端口：

- TCP：0.0.0.0:5000（手机端接入）
- WebSocket：localhost:8766（PC 网页接入）

2. 目录结构图（实际仓库快照）

```
LinkFlow/
├─ main.py
├─ requirements.txt
├─ core/
│   ├─ __init__.py
│   ├─ server.py
│   ├─ protocol.py
│   ├─ monitor_service.py
│   ├─ file_service.py
│   ├─ clipboard_service.py
│   ├─ utils.py
│   └─ __pycache__/
├─ web_pc/
│   ├─ index.html
│   └─ js/
│       └─ app.js
└─ web_mobile/
    ├─ index.html
    └─ js/
        └─ mobile_app.js
```

Python 运行产物（不建议纳入版本管理）

PC 控制台入口页（通过 `ws://localhost:8766` 接入）

当前为空（可删除或补齐）

移动端 UI 原型入口页（mock 数据）

当前为空（可删除或补齐）

3. 关键入口与运行路径

3.1 Python 服务入口

- 主入口：[[main.py](#)](file:///e:/SCL_love/LinkFlow/main.py)
 - 启动 LinkFlowServer (TCP)
 - 启动 WebBridge (WebSocket)
 - 启动 ClipboardService (轮询剪贴板)
 - 创建线程 `status_push_loop`，周期性推送系统状态到 TCP 客户端与 WebSocket 客户端

辅助入口（开发/调试）：

- [[server.py](#)](file:///e:/SCL_love/LinkFlow/core/server.py) 内含 `__main__` 简易启动测试代码（仅启动 TCP 服务）。

3.2 前端入口

- PC 控制台入口：[web_pc/index.html](file:///e:/SCL_love/LinkFlow/web_pc/index.html)
 - 浏览器会创建：new WebSocket('ws://localhost:8766')
 - 接收 sys_stats 与 file_list 等消息并更新 UI
- 移动端原型入口：[web_mobile/index.html](file:///e:/SCL_love/LinkFlow/web_mobile/index.html)
 - 目前仅 mock 数据与 UI 交互（不连后端）

4. 模块职责（按“主目录/核心文件”拆解）

4.1 core/（业务内核与协议层）

core/protocol.py：协议层（二进制打包/解包）

文件：[protocol.py](file:///e:/SCL_love/LinkFlow/core/protocol.py)

- 定义 LinkFlowProtocol：
 - **包头结构**：Type(1 byte) + Length(4 bytes) + Payload(N bytes)
 - pack(msg_type, data)：将 dict/list 序列化为 JSON，str 编码为 UTF-8，其它视为二进制
 - unpack_header(header_data)：从 5 字节包头解析类型与长度
- 定义消息类型常量（文件/剪贴板/监控/控制命令）

技术职责：

- 约束“手机端 ↔ PC 内核”的消息边界与数据表示。

依赖：

- 标准库：struct, json

core/server.py：TCP 服务端（手机端接入）

文件：[server.py](file:///e:/SCL_love/LinkFlow/core/server.py)

- LinkFlowServer：
 - start()：bind/listen 并启动 _accept_loop() 线程
 - _accept_loop()：accept 连接、保存 client_socket，为每个连接启动 _handle_client() 线程
 - _handle_client()：按协议读取 header + payload，交给 _dispatch_message()
 - _dispatch_message()：按 msg_type 调用 MonitorService / FileService 完成具体业务

- `send_to_client()`：主动向手机端推送数据包

技术职责：

- 负责“手机端连接生命周期管理 + 协议读写 + 分发到业务服务”。

关键依赖：

- 内部： `core.protocol.LinkFlowProtocol`
- 运行时局部导入： `core.monitor_service.MonitorService` 、 `core.file_service.FileService` （用于避免循环引用）
- 标准库： `socket` , `threading`

core/utils.py：WebSocket 网桥（PC 网页接入）

文件：[[utils.py](file:///e:/SCL_love/LinkFlow/core/utils.py)](file:///e:/SCL_love/LinkFlow/core/utils.py)

- `WebBridge`：
 - 在独立线程中 `asyncio.run(self._start())` 启动 WebSocket 服务
 - 维护 `clients` 集合
 - 在 `register()` 内部 `async for message in websocket` 接收网页消息，并回调 `on_message_hook(data)`
 - `broadcast(data)`：提供给**非异步线程**的安全广播接口（通过 `loop.call_soon_threadsafe` 调度）

技术职责：

- 将 Python 内核的状态/文件信息以 JSON 广播给浏览器页面。
- 将浏览器请求（例如 `get_files`）回传给 Python 侧业务处理。

关键依赖：

- 第三方： `websockets`
- 标准库： `asyncio` , `json` , `threading`

core/monitor_service.py：系统监控 + 控制命令执行

文件：[[monitor_service.py](file:///e:/SCL_love/LinkFlow/core/monitor_service.py)](file:///e:/SCL_love/LinkFlow/core/monitor_service.py)

- `get_system_stats()`：CPU、内存、电池、OS 信息
- `execute_control_command(command)`：Windows 上执行静音/睡眠/息屏/重启等

技术职责：

- 为“仪表盘展示”和“应急控制”提供数据与动作。

关键依赖：

- 第三方： psutil
- 标准库： platform , os , ctypes

core/file_service.py：文件目录读取（当前为目录列表）

文件： [file_service.py](file:///e:/SCL_love/LinkFlow/core/file_service.py)

- get_directory_info(path)：扫描目录、输出 name/type/size/mtime 并排序
- read_file_chunk()：分块读取文件（为后续流式传输准备）

技术职责：

- 为手机端或网页端提供“远程文件浏览”的基础数据能力。

关键依赖：

- 标准库： os , math

core/clipboard_service.py：Windows 剪贴板监听（轮询）

文件： [clipboard_service.py](file:///e:/SCL_love/LinkFlow/core/clipboard_service.py)

- 轮询读取 CF_UNICODETEXT，检测变化后触发 on_update_callback(text)

技术职责：

- 为“PC → 手机”的剪贴板同步提供事件源（目前仅文本）。

关键依赖：

- 第三方： pywin32 （ win32clipboard ）
- 标准库： time , threading

4.2 web_pc/（PC 控制台页面：已连后端）

入口： [web_pc/index.html](file:///e:/SCL_love/LinkFlow/web_pc/index.html)

- 使用原生 WebSocket 连接 ws://localhost:8766
- 发送：

- {"type":"get_files","path":"C:/"}
- 接收：
 - {"type":"sys_stats","val":{"...}}
 - {"type":"file_list","path":...,"list":[...]}

依赖：

- 浏览器原生 API：WebSocket、DOM
- 无构建、无第三方 npm 依赖（当前）

备注：

- web_pc/js/app.js 当前为空，属于“遗留占位”或“待拆分脚本”。

4.3 web_mobile/（移动端页面：UI 原型，未连后端）

入口：[web_mobile/index.html](file:///e:/SCL_love/LinkFlow/web_mobile/index.html)

- mock mockFS 模拟文件系统
- 通过 overlay/列表渲染实现交互原型
- 未接入 core.server 或 core.utils.WebBridge

备注：

- web_mobile/js/mobile_app.js 当前为空，同样属于占位。

5. 模块间依赖关系（核心调用链）

5.1 运行时主链路

```
main.py
├─ 启动 TCP: core.server.LinkFlowServer.start()
├─ 启动 WS: core.utils.WebBridge.start_bridge()
├─ 启动剪贴板: core.clipboard_service.ClipboardService.start_watching()
├─ 周期线程: status_push_loop()
│   ├── core.monitor_service.MonitorService.get_system_stats()
│   ├── TCP 推送: LinkFlowServer.send_to_client(TYPE_SYS_STATS, stats)
│   └─ WS 广播: WebBridge.broadcast({type:"sys_stats", val:stats})
```

5.2 手机端请求链路（目录列表）

```
手机端 -> TCP -> core.server.LinkFlowServer._handle_client()
-> core.protocol.LinkFlowProtocol.unpack_header()
-> core.server.LinkFlowServer._dispatch_message(TYPE_FILE_LIST)
    -> core.file_service.FileService.get_directory_info()
    -> LinkFlowServer.send_to_client(TYPE_FILE_LIST, dir_info)
```

5.3 网页端请求链路（目录列表）

```
web_pc/index.html -> WS -> core.utils.WebBridge.register()
-> on_message_hook({"type":"get_files"})
    -> core.file_service.FileService.get_directory_info()
    -> WebBridge.broadcast({"type":"file_list", ...})
```

6. 第三方依赖与引用路径（“从哪 import、用在哪”）

当前仓库未维护有效的依赖清单（requirements.txt 为空），但代码显式依赖如下：

- psutil：在 [monitor_service.py](file:///e:/SCL_love/LinkFlow/core/monitor_service.py) import psutil
 - 用途：CPU/内存/电池采集
- websockets：在 [utils.py](file:///e:/SCL_love/LinkFlow/core/utils.py) import websockets

- 用途：WebSocket 服务端，供浏览器连接
- pywin32 (win32clipboard)：在 [clipboard_service.py]
(file:///e:/SCL_love/LinkFlow/core/clipboard_service.py) import win32clipboard
 - 用途：Windows 剪贴板读取

建议补齐的 requirements.txt （示例，需与实际版本对齐）：

```
psutil
websockets
pywin32
```

7. 环境隔离方案（现状与推荐）

7.1 现状

- 仓库中未包含 venv/conda 配置、未提供依赖锁定文件、未提供运行脚本。
- __pycache__/ 已出现在仓库目录中，说明运行产物可能被误纳入版本管理范围。

7.2 推荐做法（团队标准）

- Python：使用 venv（或 conda）隔离依赖
 - 创建 venv：python -m venv .venv
 - 激活后安装：pip install -r requirements.txt
- 将 .venv/、__pycache__/、*.pyc 写入 .gitignore
- 将端口/host/轮询周期等运行参数移到“配置层”（环境变量或配置文件），避免硬编码在 main.py

8. 部署/运行相关资源清单

当前仓库未包含部署资源（Dockerfile、systemd/service、CI pipeline 等）。现有运行要点如下：

- 运行 PC 内核：直接启动 [main.py](file:///e:/SCL_love/LinkFlow/main.py)
- 对外暴露端口：
 - TCP：5000（局域网内手机端需能访问 PC IP）
 - WS：8766（仅本机 localhost，供本机浏览器访问）
- 前端页面：

- `web_pc/index.html` 可直接打开 (`file://`) 或通过简单静态服务器托管

建议补充的部署资源 (按优先级):

- Windows: 提供 `run.ps1` 或 `run.bat` (统一启动、打印 IP/端口、检查依赖)
- 发布: 提供 `PyInstaller` 打包脚本 (生成单文件 `exe`, 降低安装成本)
- 容器化: 如需跨机部署再考虑 `Docker` (但 Windows 剪贴板/控制命令会受限)

9. 测试、文档、构建脚本覆盖情况 (现状盘点)

9.1 测试 (缺失)

- 未发现 `tests/`、`pytest` 配置或任何自动化测试。

建议最小化补齐:

- `tests/test_protocol.py`: 验证 `pack/unpack_header` 兼容性
- `tests/test_file_service.py`: 对临时目录做目录扫描排序验证

9.2 文档 (缺失)

- 未发现 `README.md`、开发指南、协议说明、接口示例。

建议补齐最小文档集合:

- `README.md`: 用途、快速启动、端口说明、FAQ
- `docs/protocol.md`: 消息类型、样例 `payload`、兼容性策略
- `docs/dev.md`: 环境搭建、调试方式、日志约定

9.3 构建脚本 (缺失)

- 未发现 `Makefile`、任务脚本、格式化/静态检查配置。

建议补齐:

- `pyproject.toml` + `ruff` / `black` (统一风格)
- `scripts/`: 启动、打包、生成版本号等

10. 潜在重构与优化点（结合当前代码可见问题）

以下条目不会改变功能目标，但能显著降低维护成本与线上风险：

1. 修复/合并 `main.py` 中重复定义的 `on_web_message`

- [\[main.py\]\(file:///e:/SCL_love/LinkFlow/main.py#L37-L67\)](#) 出现两段同名函数定义，后者会覆盖前者，导致分支逻辑丢失风险。

2. 修正 `server.py` 分发条件的结构

- [\[server.py\]\(file:///e:/SCL_love/LinkFlow/core/server.py#L61-L93\)](#) 中 `elif` 与后续 `if` 混用，建议统一为互斥分支（`elif` 链），减少误触发/未来扩展冲突。

3. 依赖管理落地

- `requirements.txt` 为空会导致环境不可复现；建议补齐并固定版本范围（至少主版本）。

4. 引入配置层（Config）

- `host/port/推送周期/允许访问路径` 等建议从 `main.py` 抽到 `config.py` 或 `.env`。

5. 协议与业务解耦

- 当前 `LinkFlowServer._dispatch_message` 内局部导入业务 `Service`，建议将分发映射表集中化（例如 `{TYPE: handler}`），避免分支膨胀。

6. 并发与连接模型升级

- `client_socket` 只保留最后一个连接；若未来需要多设备并发，需改成连接池并增加鉴权/心跳机制。

7. 安全边界补齐

- 文件访问与系统控制属于高危能力，建议加：鉴权（token/握手）、白名单路径、审计日志、命令允许列表。

8. 前端资源整理

- `web_pc/js/app.js`、`web_mobile/js/mobile_app.js` 为空：要么删除要么把内联脚本迁移进去，避免“空文件噪音”。

9. 平台兼容性声明

- `ClipboardService` 与 `execute_control_command` 明显是 Windows 方案；建议文档与代码层显式标记平台限制并做 graceful fallback。

11. 快速定位问题（常见故障点与排查入口）

- 网页打不开数据：
 - 优先确认 WebSocket：`ws://localhost:8766` 是否启动（看 Python 控制台是否打印前端网桥已就绪）
 - 查看 [\[utils.py\]\(file:///e:/SCL_love/LinkFlow/core/utils.py\)](#) 中 `_start()` 是否异常退出

- 手机端连不上：
 - 确认 TCP 端口 5000 是否被防火墙拦截
 - 确认 PC IP 与手机同一局域网
 - 查看 [\[server.py\]](#)(file:///e:/SCL_love/LinkFlow/core/server.py) 是否有 设备已连接/已断开 日志
- 文件列表异常：
 - 入口可能来自网页（WS）或手机（TCP），对照 [\[main.py\]](#)
(file:///e:/SCL_love/LinkFlow/main.py) 的 on_web_message 与 [\[server.py\]](#)
(file:///e:/SCL_love/LinkFlow/core/server.py) 的 _dispatch_message